

Travaux Dirigés

Suites, Chiffres & Boucles

Exo	Titre	Concepts clés
B	Suite Miroir	Boucle, miroir, palindrome
D	Persistence Additive	Somme chiffres, compteur
E	Suite de Collatz	Boucle, parité, conjecture
F	Nombre parfait	Diviseurs, somme, boucle
G	Decomposition en chiffres	Chiffres, puissance, identité

Nom & Prenom :	
Numero etudiant :	
Date :	

B**Suite Miroir***Construire une suite en additionnant un nombre a son miroir***Enonce**

On part d'un entier positif **N**. On construit une suite ainsi :

- On calcule **R**, le miroir (inverse) de N (exemple : $427 \rightarrow 724$)
- Le terme suivant est **N + R**
- On repete l'operation sur ce nouveau terme, jusqu'a obtenir un **palindrome**

Exemple :

$N = 57$

$57 \rightarrow 57 + 75 = 132$

$132 \rightarrow 132 + 231 = 363$ (363 est un palindrome $\rightarrow$ arret)

Travail demande

1. Ecrire un **algorithme et un programme C** qui saisit un entier $N > 0$, affiche tous les termes de la suite jusqu'a l'obtention d'un palindrome, puis affiche ce palindrome final.

D

Persistence Additive

Compter les étapes pour réduire un entier à un seul chiffre

Enonce

On appelle **persistence additive** d'un entier **N** le nombre d'étapes nécessaires pour obtenir un nombre à un seul chiffre en répétant l'opération : *remplacer le nombre par la somme de ses chiffres*.

Exemple :

$N = 199$

$199 \rightarrow 1 + 9 + 9 = 19$

$19 \rightarrow 1 + 9 = 10$

$10 \rightarrow 1 + 0 = 1$ (1 seul chiffre $\rightarrow$ arrêt)

Nombre d'étapes = 3

Travail demande

1. Écrire un **algorithme** et un **programme C** qui calcule la persistence additive d'un entier saisi par l'utilisateur.
2. Le programme affichera chaque étape intermédiaire ainsi que le nombre total d'étapes.

E

Suite de Collatz

Etudier une conjecture mathématique célèbre sur les suites entières

Enonce

La **suite de Collatz** (ou suite de Syracuse) est définie à partir d'un entier positif **N** par la règle suivante :

- Si **N** est **pair** : le terme suivant est $N / 2$
- Si **N** est **impair** : le terme suivant est $3 * N + 1$
- On s'arrête quand **N = 1**

[Rappel] La conjecture de Collatz affirme que, quel que soit l'entier de départ, la suite atteint toujours 1. Elle n'est pas encore démontrée à ce jour.

Exemple : N = 6

```
6 -> 6/2 = 3 (pair)
3 -> 3*3+1 = 10 (impair)
10 -> 10/2 = 5 (pair)
5 -> 5*3+1 = 16 (impair)
16 -> 16/2 = 8 (pair)
8 -> 8/2 = 4 (pair)
4 -> 4/2 = 2 (pair)
2 -> 2/2 = 1 (pair) -> arret

Nombre de termes (hors 1er) = 8
```

Travail demande

1. Ecrire un **algorithme** qui saisit un entier $N > 0$ et affiche tous les termes de la suite de Collatz jusqu'à atteindre 1.
2. Ecrire le **programme C** correspondant.
3. **Extension** : Compter et afficher le nombre de termes de la suite (appelé 'temps de vol' de N). Trouver quel entier entre 1 et 100 a le plus grand temps de vol.

F**Nombre Parfait***Vérifier si un entier est égal à la somme de ses diviseurs propres***Énoncé**

Un entier positif **N** est dit **parfait** s'il est égal à la somme de tous ses diviseurs stricts (c'est-à-dire tous ses diviseurs sauf lui-même).

[Rappel] Un diviseur strict de **N** est un entier **d** compris entre 1 et **N-1** tel que $N \bmod d = 0$. Les 4 premiers nombres parfaits sont : 6, 28, 496, 8128.

Exemple : N = 28

Diviseurs stricts de 28 : 1, 2, 4, 7, 14

Somme = $1 + 2 + 4 + 7 + 14 = 28$

$28 = 28 \rightarrow 28$ est un nombre parfait

Contre-exemple : N = 12

Diviseurs stricts de 12 : 1, 2, 3, 4, 6

Somme = $1 + 2 + 3 + 4 + 6 = 16$

$16 \neq 12 \rightarrow 12$ n'est pas parfait

Travail demandé

1. Écrire un **programme C** et un **algorithme** qui saisit un entier **N** et indique s'il est parfait ou non, en affichant la liste de ses diviseurs stricts et leur somme.
2. **Extension** : Afficher tous les nombres parfaits compris entre 1 et 10000.

G

Decomposition en Chiffres

Tester si un nombre est égal à la somme des puissances de ses chiffres

Enonce

Un entier **N** a **k chiffres** est appelé **nombre de Armstrong** (ou narcissique) s'il est égal à la somme de chacun de ses chiffres élevé à la puissance **k** (le nombre de chiffres de N).

[Rappel] Nombre de chiffres **k** de **N** : compter combien de fois on peut diviser **N** par 10 avant d'atteindre 0. Pour calculer d^k , utiliser une boucle ou la fonction `pow()` de `math.h`.

Exemple 1 : N = 153 (3 chiffres -> k = 3)

Chiffres : 1, 5, 3

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

$153 = 153$ -> 153 est un nombre de Armstrong

Exemple 2 : N = 9474 (4 chiffres -> k = 4)

Chiffres : 9, 4, 7, 4

$$9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474$$

$9474 = 9474$ -> 9474 est un nombre de Armstrong

Contre-exemple : N = 100 (3 chiffres -> k = 3)

Chiffres : 1, 0, 0

$$1^3 + 0^3 + 0^3 = 1 + 0 + 0 = 1$$

$1 \neq 100$ -> 100 n'est pas un nombre de Armstrong

Travail demande

1. Ecrire un **algorithme et un programme C** qui saisit un entier **N** et indique s'il est un nombre de Armstrong, en affichant les étapes du calcul.
2. **Extension** : Afficher tous les nombres de Armstrong compris entre 1 et 9999.

Bon courage !

Bamba — Algo1 Licence 1 — 2026